Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники
Дисциплина *«Тестирование программного обеспечения»*

Лабораторная работа №1
Вариант 6345

Группа: P3323
Выполнил: Мухин Н.Д.
Проверил: Миняев И. А.

Санкт-Петербург
2026г.

Описание задания

1. Для указанной функции провести модульное тестирование разложения функции в степенной ряд. Выбрать достаточное тестовое покрытие.
2. Провести модульное тестирование указанного алгоритма. Для этого выбрать характерные точки внутри алгоритма, и для предложенных самостоятельно наборов исходных данных записать последовательность попадания в характерные точки. Сравнить последовательность попадания с эталонной.
3. Сформировать доменную модель для заданного текста. Разработать тестовое покрытие для данной доменной модели

Введите вариант: 6345

1. Функция cos(x)
2. Программный модуль для работы с B деревьями (количество элементов в ключе - до 3, http://www.cs.usfca.edu/~galles/visualization/BTree.html)
3. Описание предметной области:

> В полной темноте сверкнула ослепительно яркая точка света. Она начала расползаться в стороны, превращаясь в узкий полумесяц, и через несколько секунд показались два солнца: огненные светила, сжигающие белым пламенем черный край горизонта. Яркие цветные сполохи струились сквозь разреженную атмосферу.

Выполнение

https://github.com/n-mukhin/software_testing

Задание 1: cos(x)

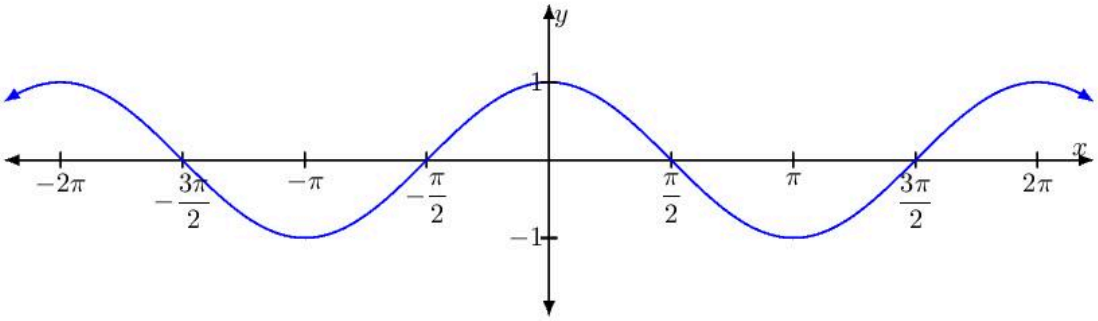

Рисунок 1 - график функции с табличными значениями на осях

1. Анализируем работу функции косинуса на граничных и некорректных значениях. Исследуем поведение функции при передаче специальных значений аргумента. Проверяем, что при x = NaN, x = +∞, x = −∞ функция возвращает NaN, а при недопустимом значении параметра точности n возникает исключение.

   Значения:
   x = NaN, +∞, −∞
   n = 0, −5

2. Рассматриваем обработку пользовательского ввода. Исследуем реакцию программы на некорректные строковые значения. Проверяем, что при вводе не числовых данных выводится сообщение об ошибке.

   Значения:
   x = "abc"
   n = "abc"

3. Проводим проверку поведения программы при отсутствии аргументов. Исследуем запуск без входных данных. Убеждаемся, что в этом случае выводится инструкция по использованию.

   Значения: аргументы отсутствуют

4. Выполняем проверку корректности вычисления функции косинуса. Исследуем значения функции в стандартных и произвольных точках. Сравниваем результат с эталонной функцией cos(x) с заданной точностью.

   Значения:
   x = 0
   x = π/6
   x = π/4
   x = π/3

x = π/2
x = π
x = 3π/2
x = 2π
x = −π/3
x = 10
x = −10

5. Изучаем свойство периодичности функции косинуса. Исследуем изменение значения функции при сдвиге аргумента на период. Проверяем выполнение равенства cos(x) = cos(x ± 2πk).

   Значения:
   x = 0
   x = π/6
   x = π/4
   x = π/3
   x = 1
   x = −2.5
   x = 10

6. Проверяем работу функции на случайных входных данных. Исследуем поведение в широком диапазоне значений аргумента. Сопоставляем результаты с эталонной реализацией.
   Значения:
   x ∈ [−50; 50] (20 случайных значений)

Element	Class, %	Method, %	Line, %	Branch, %
all	100% (2/2)	100% (3/3)	100% (27/27)	93% (15/16)
utils	100% (1/1)	100% (2/2)	100% (14/14)	90% (9/10)
Cosine	100% (1/1)	100% (2/2)	100% (14/14)	90% (9/10)
MainTask1	100% (1/1)	100% (1/1)	100% (13/13)	100% (6/6)

Рисунок 2 - тестовое покрытие файла Cosine.java и MainTask.java

В результате тестирования достигнуто полное покрытие классов, методов и строк кода (100%). Это означает, что реализация функции косинуса корректно работает для табличных значений (0, π/6, π/4, π/3, π/2, π, 2π), произвольных и отрицательных аргументов, а также сохраняет периодичность и устойчивость при некорректных входных данных (NaN, бесконечность, неверное n).

Задание 2: B-Tree

1. Рассматривается реализация B-дерева 2-3-4: самобалансирующейся структуры данных, в которой каждый узел может содержать от 1 до 3 ключей и до 4 потомков. Тестирование направлено на проверку основных операций: вставки, поиска, удаления, обхода, а также сохранения упорядоченности и корректной структуры дерева после изменений.
2. Анализируем операции вставки, поиска и обхода дерева. Исследуем формирование отсортированной структуры после добавления элементов и корректность поиска. Проверяем, что после вставки ключей результат обхода возвращает упорядоченную последовательность, а поиск корректно определяет наличие и отсутствие элементов.

   Значения:
   10, 15, 20, 30
   40, 10, 30, 20, 50, 60
   поиск: 20, 40

3. Рассматриваем удаление ключей из дерева и обработку граничных случаев. Исследуем удаление листовых и внутренних узлов, удаление единственного элемента и попытку удаления отсутствующего ключа. Проверяем, что структура дерева сохраняется корректной, используется замена внутреннего ключа преемником, а при ошибке возникает исключение.

   Значения:
   10, 20, 30, 15
   10, 20, 5, 6, 12, 30, 7, 17
   20, 10, 30, 25, 40
   удаление: 15, 10, 20, 30

4. Изучаем обработку дублирующихся значений и работу программы в целом. Исследуем поведение при повторной вставке одинаковых ключей и выполнении команд через интерфейс. Проверяем, что

дубликаты игнорируются, а основные сценарии (вставка, поиск, удаление, вывод) выполняются корректно.
Значения:
10, 20, 10, 20, 30
10, 20, 5, print
10, 20, ?10, ?30
10, 20, 30, -20, print
abc, 10

5. Проводим нагрузочное тестирование. Исследуем большое количество последовательных операций вставки и удаления. Проверяем, что дерево сохраняет корректное состояние после каждой операции и в итоге становится пустым.

   Значения:
   вставка: 0...99
   удаление: 0...99

Element	Class, %	Method, %	Line, %	Branch, %
all	100% (3/3)	100% (25/25)	97% (141/1...	94% (96/1...
BTree	100% (2/2)	100% (20/2...	100% (121/...	95% (86/90)
MainTask2	100% (1/1)	100% (5/5)	83% (20/24)	83% (10/12)

Рисунок 3 - тестовое покрытие файла BTree.java и MainTask2.java

В результате тестирования достигнуто полное покрытие классов и методов (100%), а покрытие строк кода составляет 97%, что подтверждает практически полную проверку реализации B-дерева 2-3-4, включая операции вставки, поиска, удаления и балансировки; при этом ключевой класс BTree покрыт на 100% по строкам и на 95% по ветвлениям, а общее покрытие ветвлений 94% указывает лишь на незначительное количество непроверенных сценариев, не влияющих на основную функциональность, что в целом свидетельствует о высокой надёжности и корректности реализации.

Задание 3: Описание предметной области

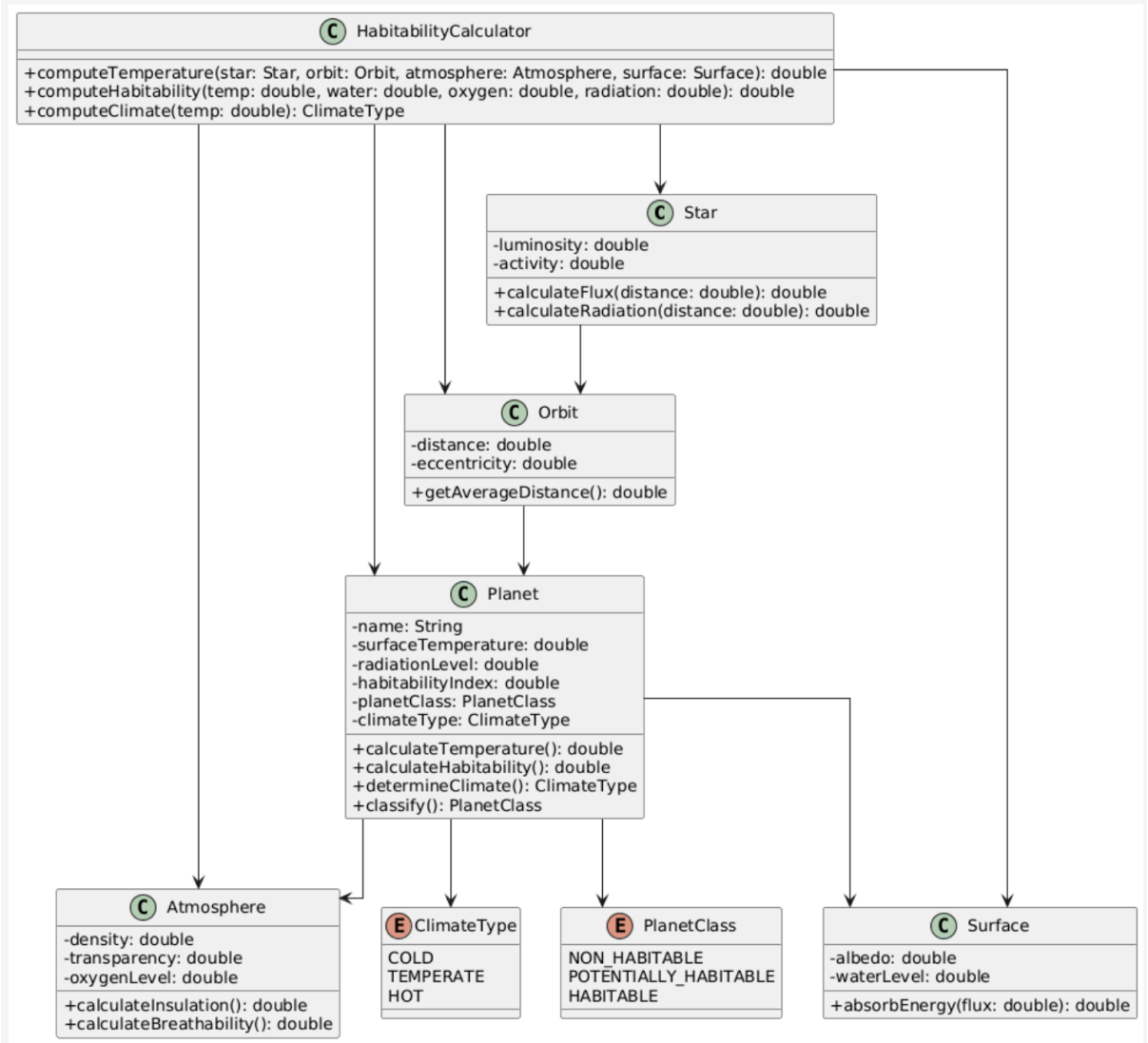

Рисунок 4 - UML Диаграмма предметной области

1. Поток излучения: $F = \dfrac{L}{4\pi d^2}$

2. Поглощённый поток: $F_{abs} = F(1 - A)$

3. Равновесная температура: $T = \left(\dfrac{F_{abs}}{\sigma}\right)^{\frac{1}{4}}$

4. Температура с учётом атмосферы: $T' = T(1 + \alpha)$

5. Ослабление радиации атмосферой: $R = \dfrac{R_0}{1 + \rho}$

6. Индекс пригодности: $H = \dfrac{W \cdot O}{1 + |T' - 22| + R}$

Рисунок 5 - используемые формулы

Выводы

В ходе выполнения лабораторной работы были освоены принципы разработки и тестирования с использованием JUnit 5. Реализованы юнит-тесты для проверки объектной модели, включая тестирование отдельных компонентов, параметризованные тесты и проверку логики взаимодействия между объектами. Получен практический опыт обеспечения корректности работы системы за счёт тестового покрытия.